

4.3 앙상블 학습(Ensemble Learning)

목차

1. 앙상블 학습 개요
 1. 앙상블 학습이란
 2. 앙상블 학습의 유형
 3. 보팅과 배깅의 차이
2. 보팅(Voting)의 두 가지 방식
 1. 하드 보팅(Hard Voting)
 2. 소프트 보팅(Soft Voting)
3. VotingClassifier 실습 - 위스콘신 유방암 예측
 1. 데이터 로딩
 2. 개별 모델과 보팅 분류기 생성
 3. 학습·예측·평가
4. 핵심 요약 및 머신러닝 활용 포인트

1. 앙상블 학습 개요

1.1 앙상블 학습이란

앙상블 학습(Ensemble Learning)은 여러 개의 분류기(Classifier)를 생성하고 그 예측을 결합하여, 하나의 분류기를 사용할 때보다 더 정확한 최종 예측을 도출하는 기법입니다. 어려운 문제를 **한 명의 전문가보다 여러 명의 집단 지성**으로 푸는 것에 비유할 수 있습니다.

이미지·영상·음성 등 비정형 데이터 분류는 딥러닝이 우세하지만, **정형 데이터 분류에서는 앙상블이 압도적으로 뛰어난 성능**을 보입니다. 캐글 같은 데이터 분석 대회의 상위권은 대부분 앙상블(특히 XGBoost, LightGBM)이 차지합니다.

■ 앙상블이 성능을 높이는 원리

서로 다른 유형의 알고리즘은 **각기 다른 방식으로 실수**합니다. 한 모델이 틀린 데이터를 다른 모델이 맞히는 경우가 많으므로, 여러 모델의 의견을 모으면 **개별 오류가 상쇄**되어 전체적으로 더 안정적이고 정확한 예측이 됩니다. 핵심은 개별 모델이 **서로 다양(diverse)해야** 한다는 점입니다. 똑같은 모델을 여러 개 모으면 효과가 없습니다.

1.2 앙상블 학습의 유형

유형	핵심 개념	대표 알고리즘
보팅 (Voting)	서로 다른 알고리즘을 가진 분류기들이 같은 데이터 세트 를 학습한 뒤 투표로 결정	VotingClassifier
배깅 (Bagging)	같은 알고리즘 의 분류기들이 서로 다르게 샘플링된 데이터 를 학습한 뒤 투표로 결정	RandomForest
부스팅 (Boosting)	여러 분류기가 순차적으로 학습하며, 앞선 분류기가 틀린 데이터에 가중치 를 부여해 다음 분류기가 집중 학습	AdaBoost, GBM, XGBoost, LightGBM
스태킹 (Stacking)	여러 모델의 예측 결과 를 다시 피쳐로 삼아 최종 메타 모델이 학습	직접 구현 / StackingClassifier

1.3 보팅과 배깅의 차이

구분	보팅(Voting)	배깅(Bagging)
알고리즘	서로 다름 (예: 로지스틱 회귀 + KNN + SVM)	모두 같음 (예: 결정 트리 100개)
학습 데이터	동일한 전체 데이터	부트스트래핑 으로 각기 다르게 샘플링
다양성 확보	알고리즘의 차이에서	데이터 샘플의 차이에서

■ 부트스트래핑(Bootstrapping)

원본 데이터에서 **중복을 허용하여** 여러 개의 샘플 데이터 세트를 만드는 방식입니다. 배깅(Bagging)이라는 이름 자체가 **Bootstrap Aggregating**의 줄임말입니다. 중복을 허용하므로 어떤 데이터는 여러 번 뽑히고, 어떤 데이터는 한 번도 뽑히지 않아 자연스럽게 **모델마다 다른 관점**이 생깁니다.

2. 보팅(Voting)의 두 가지 방식

2.1 하드 보팅(Hard Voting)

각 분류기가 예측한 **결과 레이블**을 다수결로 결정하는 방식입니다. 4개 분류기 중 3개가 클래스 1, 1개가 클래스 2를 예측했다면 최종 예측은 **클래스 1**이 됩니다. 일반 선거의 다수결과 동일한 원리입니다.

2.2 소프트 보팅(Soft Voting)

각 분류기가 예측한 **레이블별 확률**을 평균하여, 가장 높은 평균 확률을 가진 레이블을 선택하는 방식입니다.

분류기	클래스 0 확률	클래스 1 확률
분류기 1	0.7	0.3
분류기 2	0.2	0.8
분류기 3	0.8	0.2
분류기 4	0.9	0.1
평균	0.65	0.35

위 예시에서 최종 예측은 평균 확률이 더 높은 **클래스 0**이 됩니다.

■ 일반적으로 소프트 보팅이 더 우수하다

하드 보팅은 **0.51의 확신과 0.99의 확신을 똑같이 한 표**로 취급합니다. 반면 소프트 보팅은 **확신의 강도까지 반영**하므로 정보 손실이 적고, 실제로 예측 성능이 더 좋은 경우가 많습니다. 그래서 **보팅 앙상블에서는 일반적으로 소프트 보팅을** 사용합니다.

단, 소프트 보팅을 쓰려면 개별 분류기가 모두 `predict_proba()` 메서드를 지원해야 합니다.

3. VotingClassifier 실습 - 위스콘신 유방암 예측

3.1 데이터 로딩

```
import pandas as pd

from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

cancer = load_breast_cancer()

data_df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
data_df.head(3)
```

줄

코드

상세 설명

3	<code>from sklearn.ensemble import VotingClassifier</code>	<code>sklearn.ensemble</code> 모듈에서 보팅 분류기 를 임포트합니다. 앙상블 관련 클래스는 모두 이 모듈에 있습니다.
4~5	<code>LogisticRegression / KNeighborsClassifier</code>	보팅에 참여할 두 개의 서로 다른 알고리즘 을 임포트합니다. 선형 모델과 거리 기반 모델로, 성격이 달라 앙상블 효과를 기대할 수 있습니다.
6	<code>from sklearn.datasets import load_breast_cancer</code>	위스콘신 유방암 데이터 를 로딩하는 함수입니다. 종양의 크기·모양 등 30개 피쳐 로 종양이 악성(malignant) 인지 양성(benign) 인지 예측하는 이진 분류 문제입니다.
10	<code>cancer = load_breast_cancer()</code>	데이터를 Bunch 객체로 로딩합니다. 전체 569건 × 30피쳐 입니다.
12	<code>data_df = pd.DataFrame(cancer.data, columns=cancer.feature_names)</code>	피쳐 데이터를 DataFrame으로 변환해 구조를 확인합니다.
13	<code>data_df.head(3)</code>	상위 3건을 출력합니다. 피쳐가 30개라 지면상 앞쪽 6개만 표시했습니다.

▶ 실행 결과 (앞 6개 컬럼만 표시)

```
mean radius  mean texture  mean perimeter  mean area  mean smoothness  mean compactness
0      17.99      10.38      122.8      1001.0      0.11840      0.27760
1      20.57      17.77      132.9      1326.0      0.08474      0.07864
2      19.69      21.25      130.0      1203.0      0.10960      0.15990
```

```
data shape: (569, 30)  target_names: ['malignant' 'benign']
```

■ 데이터 특성

피쳐 값의 범위가 **0.08(smoothness)부터 1326(area)까지** 매우 다릅니다. KNN처럼 **거리 기반** 알고리즘은 원래 스케일링이 필요하지만, 본 예제는 앙상블 개념 설명이 목적이라 생략되어 있습니다. 실무에서는 `StandardScaler`를 적용하면 KNN 성능이 더 올라갑니다.

3.2 개별 모델과 보팅 분류기 생성

```
# 개별 모델은 로지스틱 회귀와 KNN 임.
lr_clf = LogisticRegression(solver='liblinear')
knn_clf = KNeighborsClassifier(n_neighbors=8)

# 개별 모델을 소프트 보팅 기반의 앙상블 모델로 구현한 분류기
vo_clf = VotingClassifier( estimators=[('LR',lr_clf),('KNN',knn_clf)] , voting='soft' )

X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target,
                                                    test_size=0.2 , random_state= 156)
```

줄	코드	상세 설명
2	<code>lr_clf = LogisticRegression(solver='liblinear')</code>	로지스틱 회귀 객체입니다. <code>solver='liblinear'</code> 는 작은 데이터에 적합한 최적화 알고리즘 으로, 스케일링하지 않은 데이터에서 발생하는 수렴 경고 를 방지합니다.
3	<code>knn_clf = KNeighborsClassifier(n_neighbors=8)</code>	KNN 분류기입니다. <code>n_neighbors=8</code> 은 가장 가까운 8개 이웃의 다수결 로 클래스를 결정한다는 의미입니다.

6	VotingClassifier(estimators=[('LR',lr_clf), ('KNN',knn_clf)], voting='soft')	이 실습의 핵심입니다. • estimators : (이름, 분류기 객체) 튜플의 리스트를 전달합니다. 이름은 결과 확인이나 파라미터 튜닝 시 참조용으로 쓰이며 임의로 지어도 됩니다. • voting='soft' : 소프트 보팅을 지정합니다. 기본값은 'hard' 이므로 반드시 명시해야 확률 평균 방식이 적용됩니다.
8~9	train_test_split(..., test_size=0.2, random_state=156)	569건을 학습 455건 / 테스트 114건으로 분리합니다. 시드를 고정해 결과를 재현합니다.

3.3 학습·예측·평가

```
# VotingClassifier 학습/예측/평가.
vo_clf.fit(X_train , y_train)
pred = vo_clf.predict(X_test)
print('Voting 분류기 정확도: {0:.4f}'.format(accuracy_score(y_test , pred)))

# 개별 모델의 학습/예측/평가.
classifiers = [lr_clf, knn_clf]
for classifier in classifiers:
    classifier.fit(X_train , y_train)
    pred = classifier.predict(X_test)
    class_name= classifier.__class__.__name__
    print('{0} 정확도: {1:.4f}'.format(class_name, accuracy_score(y_test , pred)))
```

줄	코드	상세 설명
2	vo_clf.fit(X_train, y_train)	보팅 분류기를 학습합니다. 내부적으로 등록된 두 개별 모델이 각각 전체 학습 데이터로 학습됩니다. VotingClassifier도 일반 Estimator와 동일한 <code>fit()</code> 인터페이스를 씁니다.
3	pred = vo_clf.predict(X_test)	예측 시 두 모델이 각각 <code>predict_proba()</code> 로 확률을 내고, 그 평균이 가장 높은 클래스가 최종 예측이 됩니다.
4	accuracy_score(y_test, pred)	보팅 분류기의 정확도를 출력합니다.
7	classifiers = [lr_clf, knn_clf]	비교를 위해 개별 모델을 리스트에 담습니다. 반복문으로 처리하기 위한 준비입니다.
8~9	for classifier in classifiers: classifier.fit(...)	각 모델을 단독으로 학습·예측합니다. 사이킷런의 Estimator 인터페이스가 통일되어 있어 반복문 하나로 여러 알고리즘 처리가 가능합니다.
11	class_name = classifier.__class__.__name__	객체의 클래스 이름을 문자열로 얻습니다. 하드코딩 없이 모델명을 출력할 수 있는 유용한 기법입니다.
12	print('{0} 정확도: {1:.4f}'...)	모델명과 정확도를 함께 출력합니다.

▶ 실행 결과

```
Voting 분류기 정확도: 0.9561
LogisticRegression 정확도: 0.9474
KNeighborsClassifier 정확도: 0.9386
```

■ 결과 해석 - 앙상블 효과 확인

모델	정확도	비고
Voting (소프트)	0.9561	두 모델보다 모두 높음
LogisticRegression	0.9474	개별 모델 중 우수
KNeighborsClassifier	0.9386	개별 모델 중 낮음

보팅 분류기가 개별 모델 둘 다보다 높은 정확도를 기록했습니다. 114건 기준으로 로지스틱 회귀보다 1건, KNN보다 2건을 더 맞힌 것입니다. 두 모델이 서로 다른 데이터에서 실수했고, 확률 평균이 그 오류를 보정했음을 보여 줍니다.

⚠ 앙상블이 항상 개별 모델보다 낫지는 않다

보팅으로 여러 분류기를 결합해도 무조건 성능이 향상되는 것은 아닙니다. 데이터의 특성과 개별 모델의 조합에 따라 가장 좋은 단일 모델보다 낮게 나올 수도 있습니다. 특히 성능이 지나치게 나쁜 모델을 포함시키면 전체 평균을 끌어내립니다.

또한 이 결과는 테스트 114건에 대한 단 한 번의 측정이므로, 실무에서는 `cross_val_score` 로 교차 검증하여 판단해야 합니다.

■ 참고 - 하드 보팅으로 바꿔 보기

`voting='hard'` 로 변경하면 확률 대신 예측 레이블의 다수결로 동작합니다. 다만 분류기가 2개일 때는 동점 상황이 발생할 수 있는데, 이 경우 사이킷런은 먼저 등록된 분류기(여기서는 LR)의 예측을 따릅니다. 그래서 분류기 개수는 홀수로 두는 것이 일반적입니다.

4. 핵심 요약 및 머신러닝 활용 포인트

4.1 핵심 개념 요약

개념	핵심 내용
앙상블 학습	여러 분류기의 예측을 결합해 더 정확한 최종 예측을 만드는 기법
보팅	서로 다른 알고리즘 + 같은 데이터
배깅	같은 알고리즘 + 서로 다른 샘플 데이터 (랜덤 포레스트)
부스팅	순차 학습, 이전 모델의 오류에 가중치 부여 (GBM, XGBoost)
하드 보팅	예측 레이블의 다수결
소프트 보팅	예측 확률의 평균 — 일반적으로 더 우수

4.2 핵심 API 요약표

API / 파라미터	설명	주의 사항
<code>VotingClassifier()</code>	보팅 앙상블 분류기	<code>sklearn.ensemble</code> 에 위치
<code>estimators=[('이름', 객체), ...]</code>	참여할 분류기 목록	(이름, 객체) 튜플의 리스트 형태

<code>voting='soft'</code>	확률 평균 방식	기본값은 'hard'이므로 명시 필요 개별 모델이 <code>predict_proba()</code> 지원해야 함
<code>voting='hard'</code>	레이블 다수결 방식	분류기 개수는 홀수 권장
<code>weights=[2, 1]</code>	분류기별 가중치	성능 좋은 모델에 더 큰 비중 부여 가능
<code>obj.__class__.__name__</code>	객체의 클래스명 문자열	여러 모델 비교 출력 시 유용

4.3 머신러닝 활용 포인트

- **다양성이 핵심** : 비슷한 성격의 모델만 모으면 효과가 없습니다. **선형 모델 + 트리 모델 + 거리 기반 모델**처럼 성격이 다른 조합이 좋습니다.
- **소프트 보팅을 기본으로** : 확률 정보를 활용하므로 대부분의 경우 하드 보팅보다 우수합니다.
- **개별 모델도 함께 평가** : 앙상블만 측정하면 **실제로 개선되었는지** 알 수 없습니다. 본 실습처럼 개별 성능을 반드시 함께 확인하세요.
- **전처리는 개별 모델에 맞게** : KNN·로지스틱 회귀는 스케일링이 필요하지만 트리 계열은 불필요합니다. 서로 다른 전처리가 필요하면 `Pipeline` 을 각 모델에 개별 적용한 뒤 보팅에 넣습니다.
- **다음 학습 방향** : 보팅은 앙상블의 입문입니다. 실무에서 더 강력한 성능을 내는 것은 **배깅(4.4 랜덤 포레스트)**과 **부스팅(4.5 GBM, 4.6 XGBoost, 4.7 LightGBM)** 계열입니다.

⚠ 안정화 버전 참고 (scikit-learn 1.7.x 기준)

- `VotingClassifier` 의 API는 변경 없이 안정적으로 유지되고 있습니다.
- 회귀 문제에는 `VotingRegressor` 를 사용하며, 이 경우 예측값의 **평균**을 취합니다.
- `LogisticRegression` 에서 `solver` 를 지정하지 않으면 기본값 `'lbfgs'` 가 적용되어, 스케일링하지 않은 본 데이터에서는 **`ConvergenceWarning`**이 발생할 수 있습니다.
- 사이킷런 0.22부터 스택킹 전용 클래스 `StackingClassifier` 도 제공됩니다(4.11장 참고).